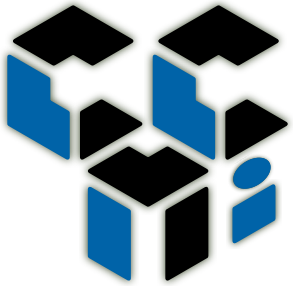
OntoUML: Shrnutí OntoUML, Anti-Patterns, převod na UML





FAKULTA INFORMAČNÍCH TECHNOLOGIÍ ČVUT V PRAZE

BI-KOM: Konceptuální modelování (Cvičení #6)

Zimní semestr 2026/2027 (B261)

Plán 6. cvičení

1. Shrnutí aktivity z minula
2. Shrnutí OntoUML
3. OntoUML Anti-Patterns
4. Převod OntoUML na UML
5. Příklad na procvičení (za bod)
6. Úkoly na příště

Shrnutí aktivity z minula

- Multiplicita je u všech asociací povinná!
- Vyplněný/nevyplněný kosočtverec je na straně celku a závisí na maximální multiplicitě.
- Essential/inseparable určují nezaměnitelnost části/celku.
- Pozor na rozdíl mezi Quantity, Collective a Functional complex.

Shrnutí OntoUML

- OntoUML je UML profil obohacující Class Diagram o pojmy z UFO formou stereotypů
- Založený na filozofii a kognitivních vědách - princip identity, rigidity
- Použité stereotypy rozšiřují význam obecných *tříd* a *vztahů*
- Ke stereotypům se dle jejich kategorizace váží specifická pravidla
  - Co může být podtřídou čeho a co ne
  - Mezi jakými typy lze použít daný typ vztah (např. «subQuantityOf»)
  - Že «Role» musí mít povinný vztah
  - ...

Rigidní sortály

- «Kind» = základní rigidní sortál poskytující identitu (*identity provider*)
  - «Relator» = rigidní sortál poskytující identitu, *truthmaker* «material» vztahu
  - «Collective» = rigidní sortál poskytující identitu, celek se členy stejného druhu
  - «Quantity» = rigidní sortál poskytující identitu, celek bez rozlišení jednotlivých členů (např. tekutiny, sypké materiály, maziva a další)
- «SubKind» = rigidní sortál dědící identitu

Anti-rigidní sortály

- Musí (stejně jako «SubKind») zdědit identitu (tranzitivně) z právě jednoho *identity provider* = každá instance musí mít **právě jednu identitu**
- «Phase» = pouze vnitřní vlastnosti, které se mohou v čase měnit
  - Vždy tvoří *phase partition*, který musí být `{disjoint, complete}`
  - Například: `Alive` a `Deceased`
- «Role» = vlastnosti závisející na další entitě, které se mohou v čase měnit
  - Vždy musí mít **povinný vztah**, který "potvrzuje", že je instance *v roli*
  - Například: `Student`, `Employee` nebo `Committee`

Non-sortály

- Nadtřídy pro společné vlastnosti entit **různých druhů** = různých principů identit
- Vždy se jedná o **abstraktní třídy** = název kurzívou
- «Category» = rigidní non-sortál, pro rigidní podtypy
- «PhaseMixin» = anti-rigidní non-sortál, pro «Phase» různých druhů
- «RoleMixin» = anti-rigidní non-sortál, relačně závislý, pro «Role» různých druhů
- «Mixin» = semi-rigidní non-sortál, pro rigidní **a současně** anti-rigidní podtypy

Aspekty

- *Slabé* entity, vždy závislé a náležící svému nositeli (*bearer*), typicky významné atributy
- «Quality» = měřitelné veličiny
  - Například: `Height`, `Color`, `Density`, `Volume` (objem i hlasitost)
- «Mode» = neměřitelné veličiny a stavy
  - Například: `Symptoms`, `Skill`, `Expertise`, `Book Edition`, …

Asociace

- Stejně jako třídy, tak i vztahy musí mít v OntoUML blíže určený typ
- «formal» = vztah odvozený přímo z vlastností entit ve vztahu
- «material» = vztah odvozený z další entity, která je *truthmaker* vztahu
  - *truthmaker* = třída se stereotypem «Relator» (či podtřída)
  - «Relator» pattern využívající «Role» na obou stranách vztahu
  - multiplicity pro «material» a odpovídající «mediation» musí být konzistentní
- «containment» = vztah mezi «Quantity» a kontejnerem, vždy 1-1
- «characterization» = vztah mezi aspektem a nositelem, vždy 1 na straně nositele
- «derivation» = vztah mezi «material» a «Relator» (čárkovaná čára)

Vztahy celek-část

- Diamant na straně celku, plný = nesdílitelné části, prázdný = sdílitelné mezi více celky
- «memberOf» (**M**) = mezi členy a «Collective», vždy min. 2 na straně části
- «subCollectionOf» (**C**) = mezi «Collective», vždy 1-1 kvůli *maximalitě*
- «subQuantityOf» (**Q**) = mezi «Quantity», vždy 1-1 kvůli *maximalitě*
- «componentOf» = mezi částí a funkčním celkem

Omezení Generalization Set

- (bez omezení) = instance lze tvořit z nadtřídy a libovolné kombinace podtříd
- `{disjoint}` = instance nadtřídy nebo (xor) jedné z podtříd
- `{complete}` = instance libovolné kombinace podtříd (ne pouze nadtřídy)
- `{disjoint, complete}` = instance právě jedné z podtříd

Omezení vztahů celek-část

- `{essential}` = celek nemůže existovat bez stále stejné instance části
- `{inseparable}` = část nemůže existovat aniž by byla částí stále stejného celku
- V případě anti-rigidity chceme zajistit **neměnnost** při ztrátě a znovu-získání vlastností (např. změna fáze *tam a zpět*)
- `{immutablePart}` = podobné `{essential}`, ale pro anti-rigidní celek
- `{immutableWhole}` = podobné `{inseparable}`, ale pro anti-rigidní část

Jak kontrolujeme modely?

- Verifikace:
  - Syntaktická kontrola
- Validace:
  - Sémantická kontrola
  - Kontrola souladu s doménou
- OntoUML na rozdíl od UML dovoluje automatizovat detekci jak syntaktických, tak sémantických chyb, díky vysoké míře formalizace a expresivity jazyka.
- Kontrolu souladu s doménou je vždy nutno provádět manuálně.

Syntaktická kontrola - OpenPonk

- Kontrola na základě OntoUML specifikace.
- V OpenPonku reprezentována verifikacemi.
- Všechny syntaktické chyby je nutno opravit!

Sémantická kontrola - Anti-patterny

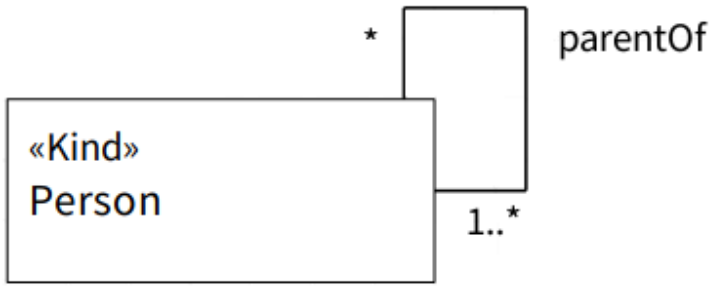
- Sémantická kontrola modelu.
- Detekce podezřelých struktur v modelu.
- V OpenPonku implementováno celkem dvacet anti-patternů.
- Výskyt anti-patternu většinou znamená chybu.
- Plná dokumentace anti-patternů na ontouml.readthedocs.io/en/latest/anti-patterns/index.html

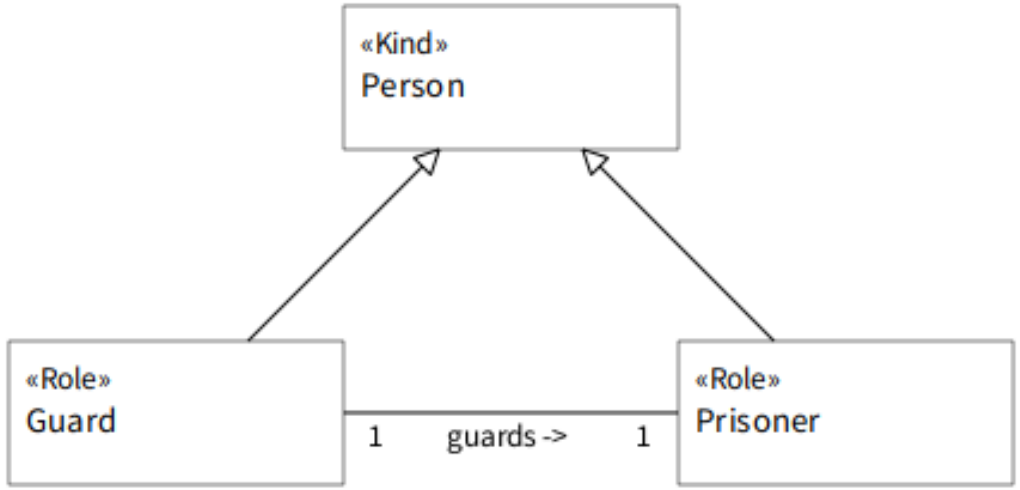
Anti-patterny - seznam

- Binary relation between overlapping types (BinOver)
- Relationally dependent phase (DepPhase)
- Free role specialization (FreeRole)
- Heterogeneous collective (HetColl)
- Mixin with the same rigidity (MixRig)
- Multiple relational dependency (MultDep)
- Relator mediating rigid types (RelRig)
- Undefined phase partition (UndefPhase)

Binary relation between overlapping types (BinOver)

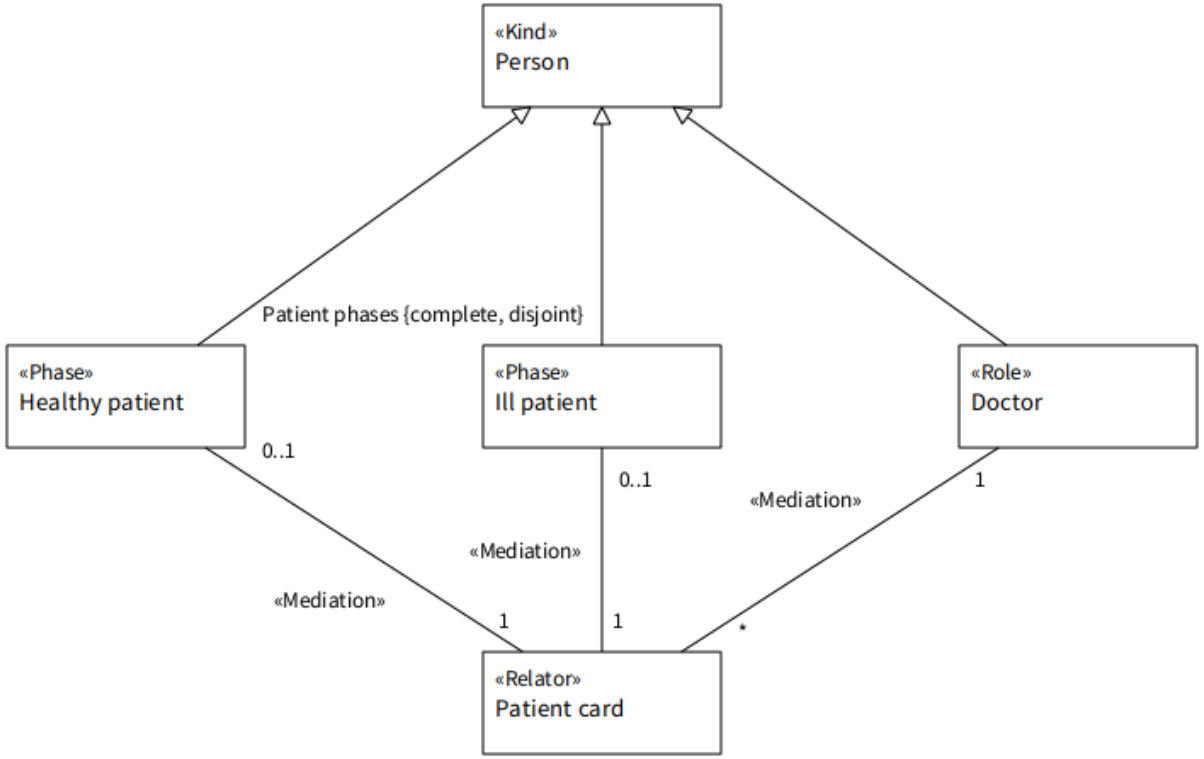
- Overlapping typy = dva typy, které mohou být instancovány stejným individuem.
- **"Na obou stranách relace může být stejné individuum".**
- Existují další "...Over" anti-patterny (PartOver, RelOver, WholeOver).





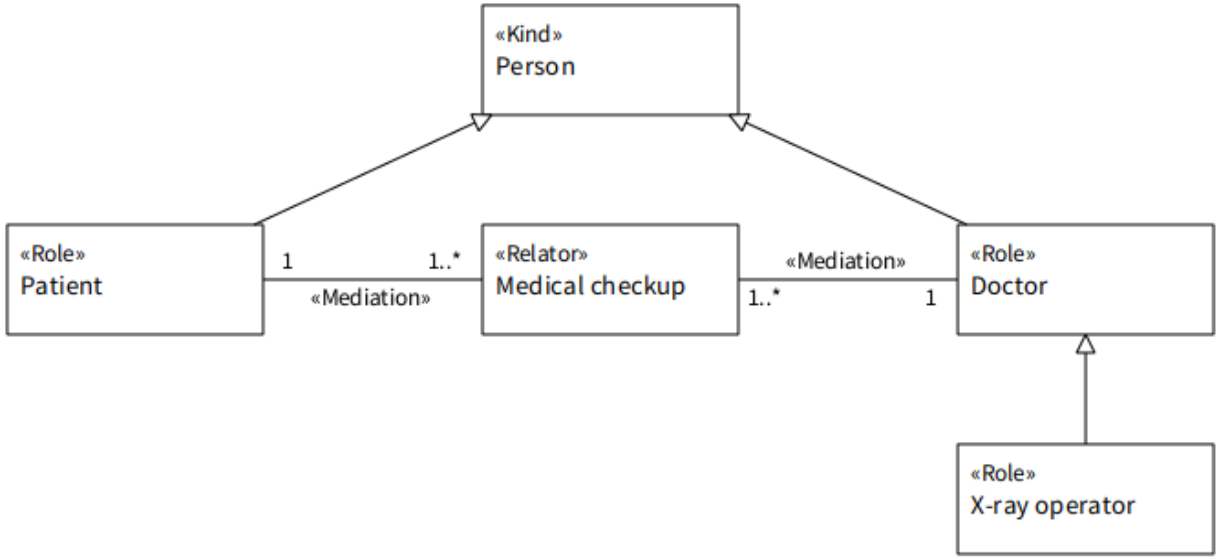
Relationally dependent phase (DepPhase)

- **Phase, která je součástí jedné nebo více asociací typu mediation.**
- Phase = závislá na změně vnitřního stavu.
- Role = závislá na existenci vnější asociace.



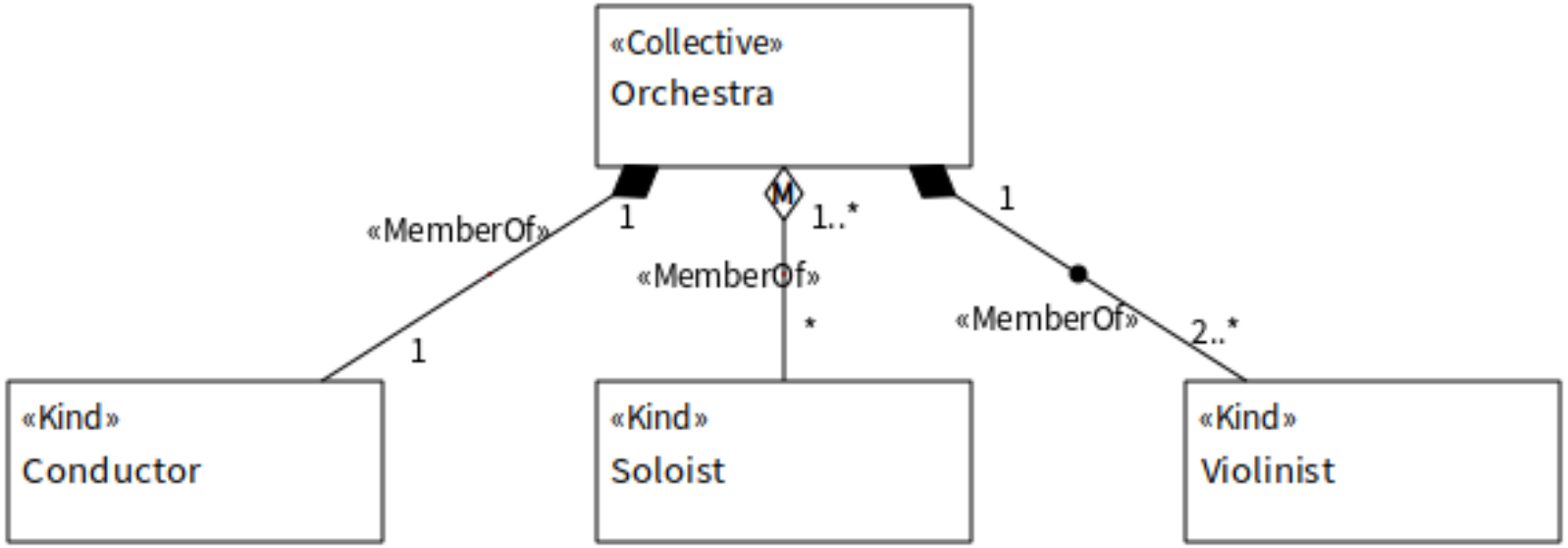
Free role specialization (FreeRole)

- Splnění relační závislosti role:
  - Role je součástí asociace stereotypované jako mediation.
  - Role je přímým podtypem rolemixinu, který je součástí mediation asociace.
  - **Role je podtypem jiné role, která má splněnou relační závislost**



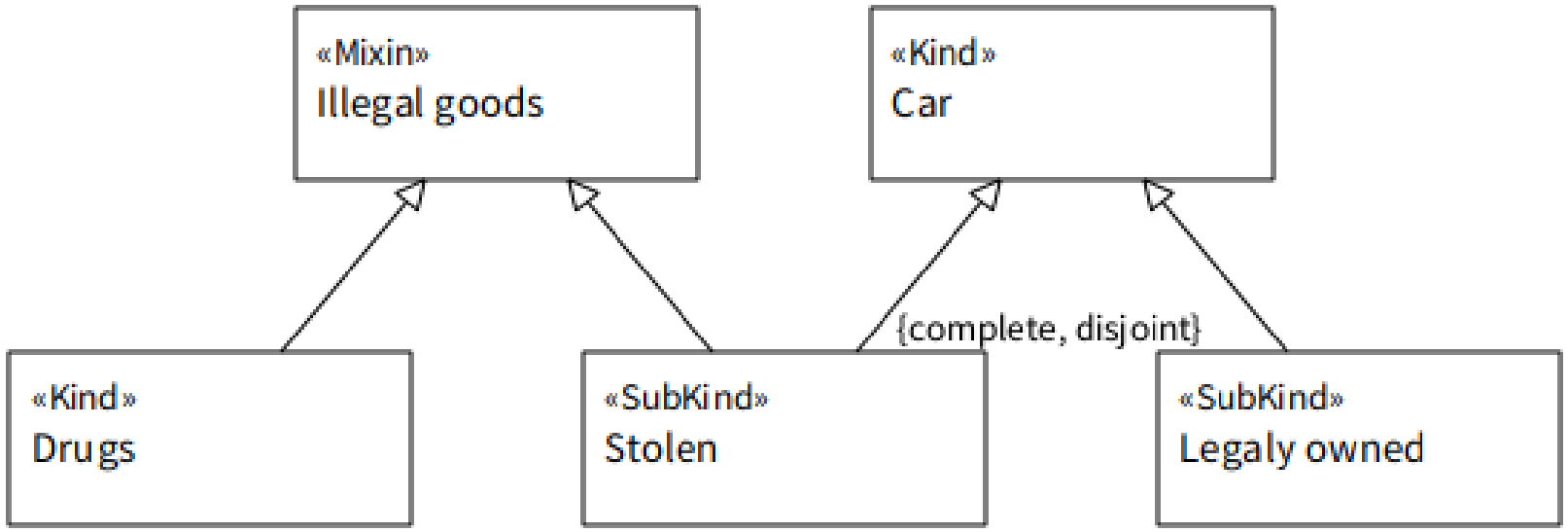
Heterogeneous collective (HetColl)

- Opakem je Homogeneous Functional Complex (HomoFunc).
- Collective = na všechny části je nahlíženo stejně.
- Funkční celek = jednotlivé části mají různé role.
- **Collective s dvěma či více rozdílnými typy částí.**



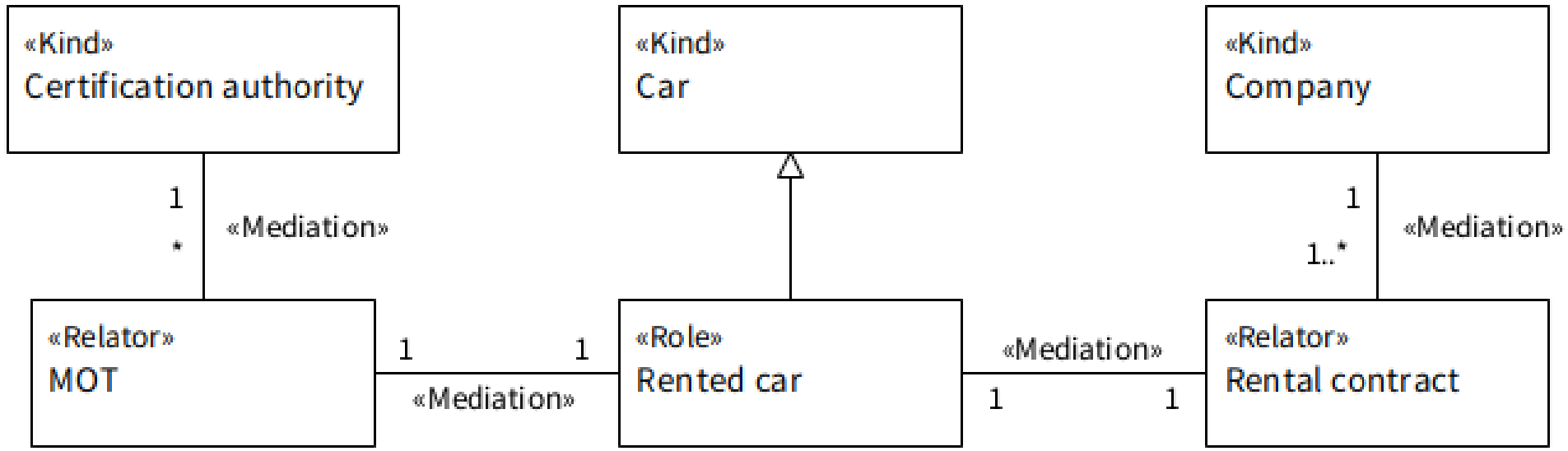
Mixin with the same rigidity (MixRig)

- Mixin může být nadtypem rigidních, anti-rigidních i semi-rigidních typů.
- Mixin musí mít minimálně dva podtypy s různou rigiditou.
- **Mixin, jehož podtypy jsou pouze rigidní, nebo pouze anti-rigidní podtypy.**



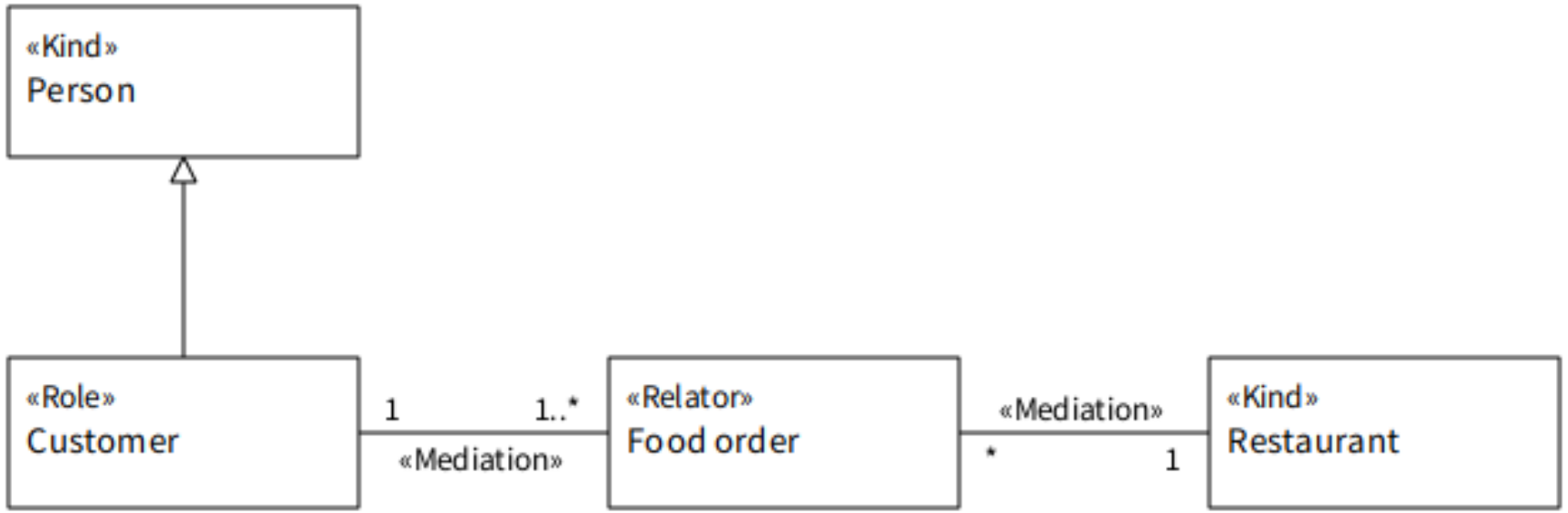
Multiple relational dependency (MultDep)

- **Typ, který je propojen pomocí mediation s dvěma a více různými relatory, které nejsou nadtypem/podtypem jeden druhého.**
- Velmi často důsledek redundance, přílišného zjednodušení, nebo špatné granularity modelu.



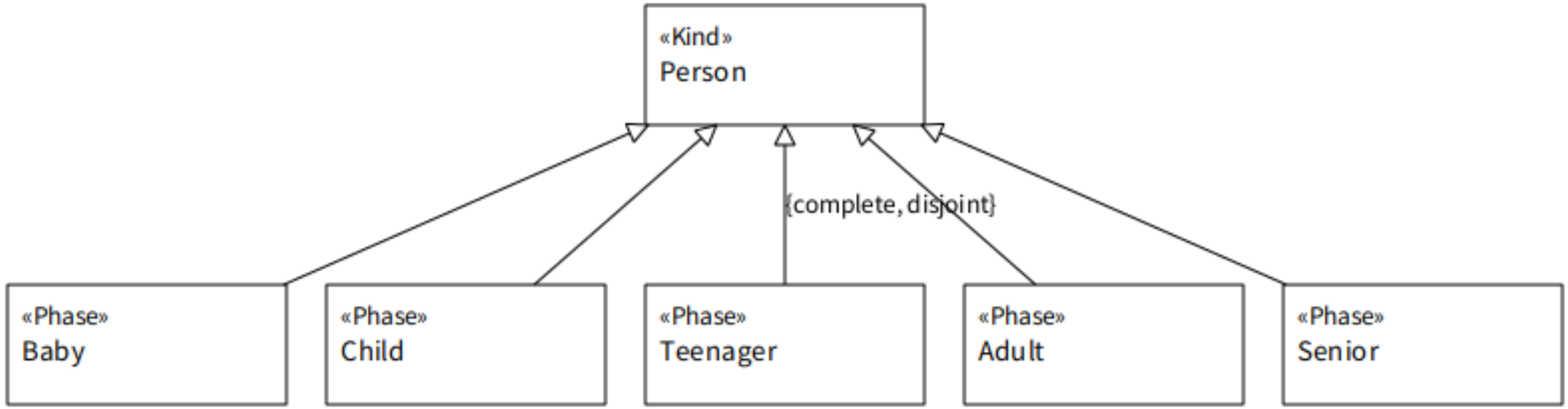
Relator mediating rigid types (RelRig)

- **Relator, přímo propojený mediation asociací s jedním nebo více rigidními typy.**
- Většinou značí chybějící roli, případně rolemixin.
- Může být též znakem toho, že rigidní typ má být stereotypován jako mode.



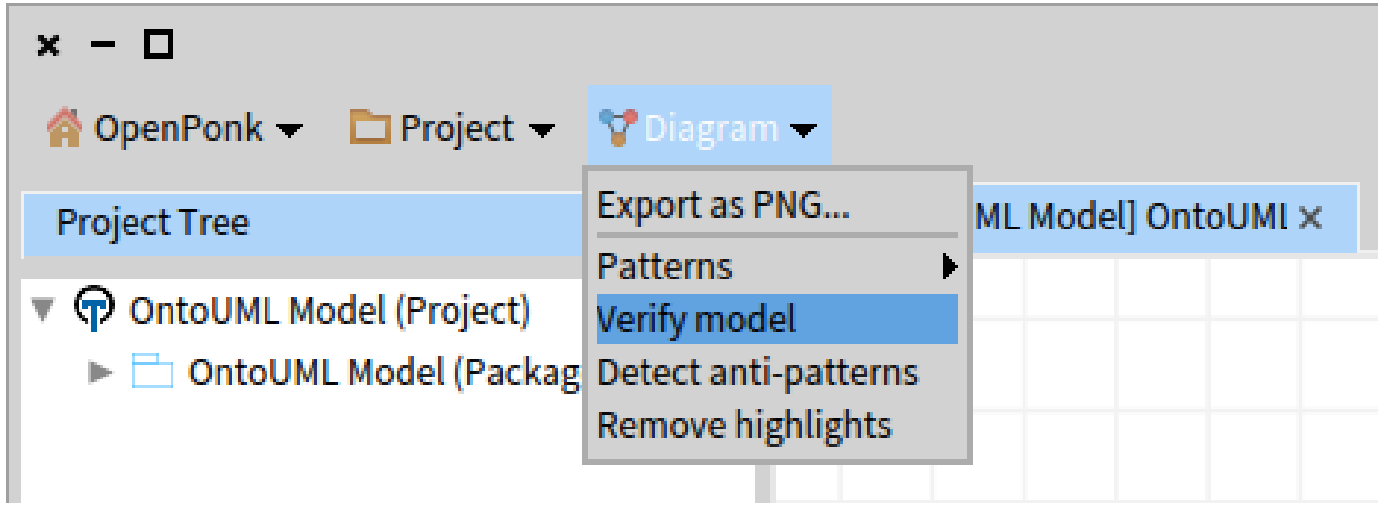
Undefined phase partition (UndefPhase)

- **Množina entit stereotypovaných jako phase, jejichž společný nadtyp nemá žádný atribut, quality ani mode.**
- Phase jsou závislé na změně vnitřního stavu souvisejícího s vlastností nadtypu.
- Tyto vlastnosti lze přiřadit přímo k jednotlivým phase.

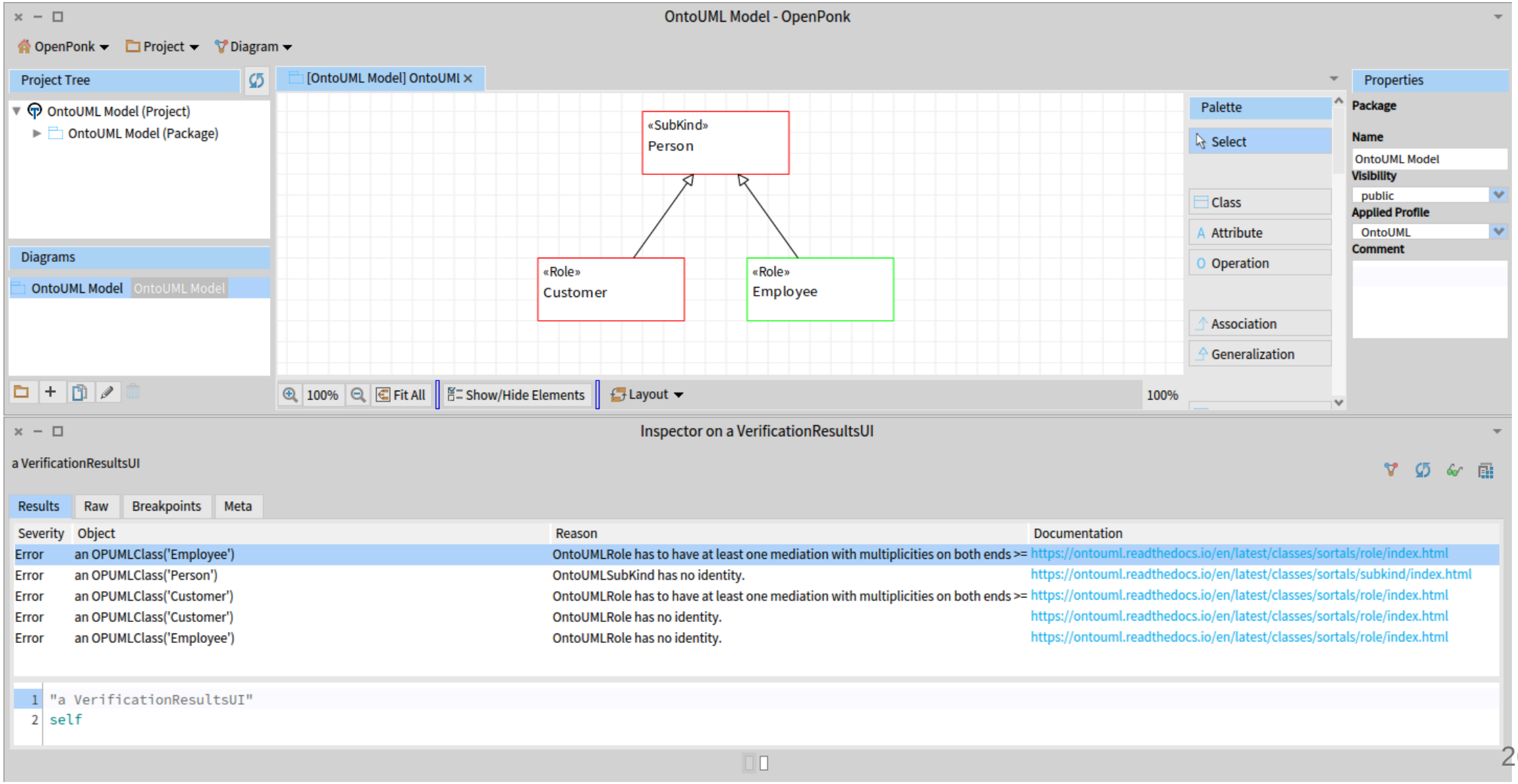


OpenPonk - verifikace a anti-patterny

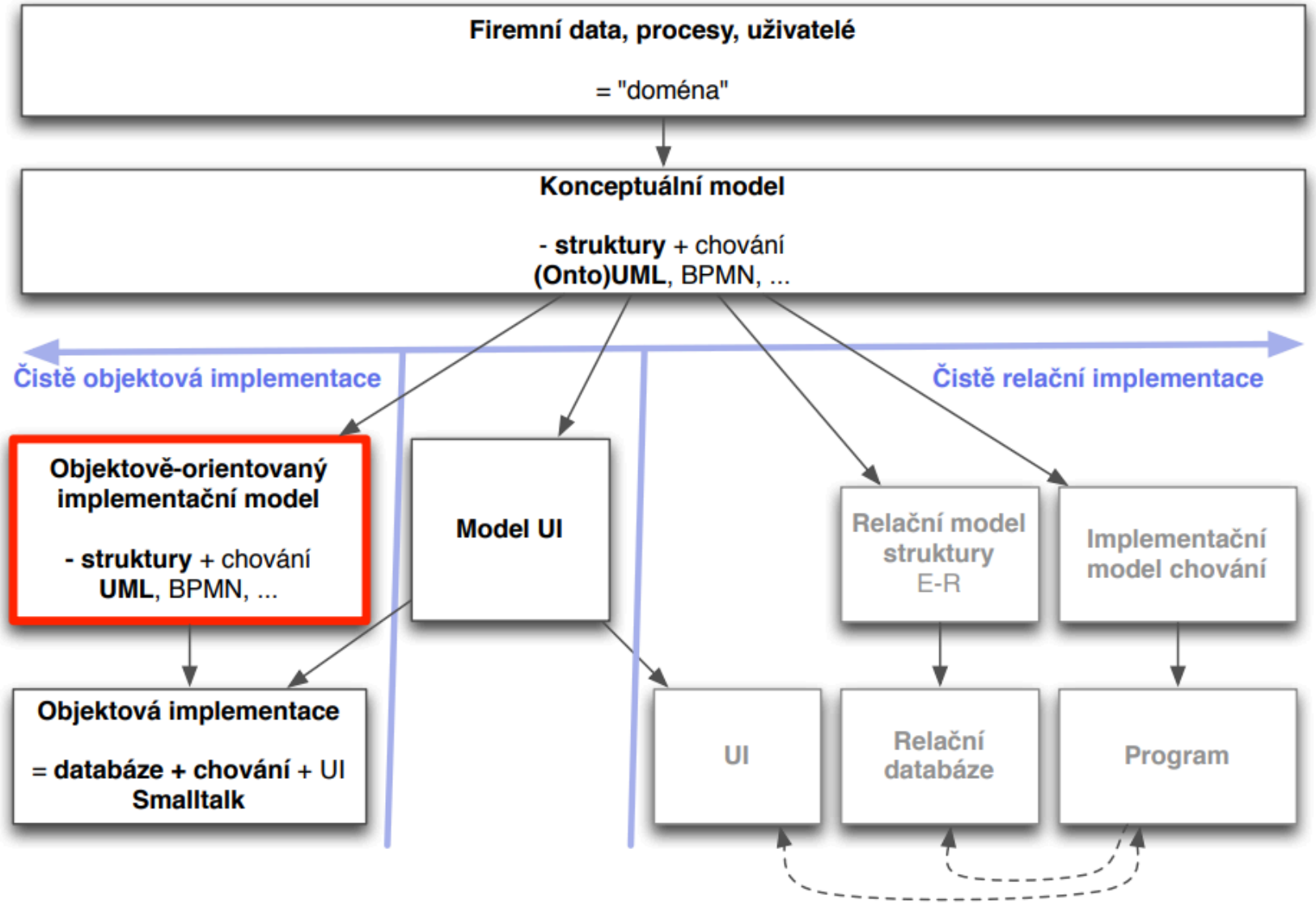
- Menu "Diagram" v toolbaru hlavního okna
- "Verify model" - syntaktická kontrola, nálezy obarví elementy na červeno
- "Detect anti-patterns" - sémantická kontrola, nálezy obarví elementy do žluta
- "Remove highlights" - vrátí elementům defaultní barvu



OpenPonk - verifikace a anti-patterny



Převod OntoUML na OO



Převod OntoUML na OO

- Konceptuální modelování slouží primárně k zachycení domény pro účely porozumění a komunikace
- Konceptuální model lze ale využít jako podklad pro datový model informačního systému
- Tímto se zabývá oblast softwarového inženýrství zvaná Model-Driven Development
- Existuje i definovaná transformace z OntoUML na objektový model
- Objektový model je méně expresivní než OntoUML a tak se přirozeně ztratí některé *detaily*

Převod OntoUML na OO: Třídy a atributy

- Typy entit z OntoUML (zachycené jako třídy se stereotypy) se běžně převádí na třídy
- Třídy si zachovávají atributy, můžeme již blíže specifikovat datové typy.
- Zajišťování povinnosti (atributů i vztahů) pomocí *soft* a *hard* metod:
  - *Hard* = předání v konstruktoru, objekt je vždy validní
  - *Soft* = objekt může být nevalidní po nějakou dobu, dokud nenastane kontrola konzistence
- U asociací (obousměrných vztahů) nutno speciálně ošetřovat
  - Zvolit vhodný směr asociace podle use-case
  - Relationship Object Pattern

Převod OntoUML na OO: Dědičnost

- Dědičnost v OntoUML funguje podstatně jinak než v běžném OO
- Při `{complete}` je nadtřída abstraktní
- `{disjoint}` je běžný případ v OO, bez něj je potřeba dělat *průnikové* třídy: skládání nebo dědění
- *Composition over Inheritance* a *Don't Repeat Yourself*

Převod OntoUML na OO: Sortály

- Zachycení identity pomocí atributů, často ale volíme (zároveň) umělé identifikátory
- Převod anti-rigidních sortálů typicky pomocí skládání
  - Role: 3 alternativy podle use-case (přístupnost a flexibilita)
  - Phase: state pattern, degradace na atribut (ztráta polymorfismu)

Převod OntoUML na OO: Non-sortály

- Abstraktní třídy pro sdílené vlastnosti
- Vede typicky na vícenásobnou dědičnost
- Často vhodné změnit dědění na skládání a delegovat na objekt non-sortálu
- V takovém případě již není abstraktní

Převod OntoUML na OO: Agregace

- Vztahy celek části podobně jako klasické asociace
- U kolektivu typicky stačí instance kolekce (v kolekci jsou prvky dle sjednocující relace)
- Lze implementovat i podkolektivy, které se dynamicky aktualizují
- Nutné implementovat omezení pomocí konstruktorů a zakázání změny celku nebo části v rámci života objektu

Převod OntoUML na OO: Shrnutí

- V OOP je dědičnost vždy rigidní = nelze ji použít pro dědičnost anti-rigidních typů – musíme ji *emulovat* skládáním.
- U systémů s jednoduchou dědičností (ST) zajišťujeme implementaci vícenásobné dědičnosti opět skládáním či speciálními konstrukty (`trait`, `mixin`).
- Některá strukturální omezení (např. povinnost v multiplicitě) zajišťujeme vhodným ošetřením v kódu metod.
- Aspekty dle potřeby implementujeme jako atributy nebo třídy.
- Implementaci rolí a fází řešíme vhodnou variantou dle situace.

Aktivita za bod

Zadání se nachází na Moodle.

Odevzdejte .opp model a náhled modelu (ideálně jako PDF nebo JPG) v: **Aktivita / Activity #6**

Midterm info

- Otevřený playground v Moodle pro učení se
- Neomezený počet pokusů
- Otevřeno na týden (přesné uzavření v Moodle)
- Stejné otázky jako budou v testu
- Pokud narazíte na problém, kontaktujte vyučující
- Midterm bude příští týden na cvičení

Úkoly do příště

- Pokračovat na semestrální práci:
  - Textový popis
  - OntoUML model
  - Omezení (co za pravidla z domény nelze vyjádřit přímo v OntoUML)
- Příští přednáška a cvičení: OCL

Konec prezentace